

统一异常处理与日志模块 — 设计文档 v1

0. 前端总览

本章为后端基础架构设计，不涉及前端界面变动。

核心目标：服务运行时的错误不再「静默吞掉」或「零散输出」，而是：1. 所有异常有统一的分层类型 → 可识别、可追踪 2. 所有日志通过统一入口输出 → 格式一致、可分级检索 3. API 错误响应结构统一 → 前端可统一处理 4. 全局拦截未捕获异常 → 不崩溃、有记录

1. 背景与问题

现状

维度	现状	问题
日志模块	services/logger.py 已存在，功能完整 (RotatingFileHandler + 控制台)	放在 services/ 下语义不合理；其他模块未统一使用
模块内部日志	6 个文件用 from backend.services.logger import get_logger	9 个文件用原始 import logging，格式/级别不统一
异常处理	70+ 处 except Exception:	大部分静默 (pass) 或只返回 str(e) 不记日志
API 错误响应	各 handler 各自处理，无统一结构	前端无法统一判断错误类型
全局异常拦截	do_GET() / do_POST() 无 try-except	未捕获异常 → 500 页面崩溃，无日志

典型问题模式

模式 A — 静默吞异常（最危险）：

```
# on_demand_stock.py:111
except Exception:
    pass          # 出问题完全不知道
```

模式 B — 只返回不记录：

```
# stock.py:19
except Exception as e:
    result['diagnosis'] = {'error': str(e)} # 前端看到错误但后端不知道
```

模式 C — 日志使用混乱：

```
# diagnosis_service.py
import logging as log          # 别名随意
log.info('...')

# panic_monitor_service.py
import logging                 # 函数内 import
logger = logging.getLogger(__name__)

# server.py
from backend.services.logger import get_logger
log = get_logger('server')
```

目标

一句话：让每个错误都有据可查，让每行日志都有迹可循。

2. 设计思想

2.1 核心理念

「三统一」架构：

- 日志入口统一 — 所有模块通过 core/logger.py 的 get_logger() 获取 logger，配置从 config.py 读取

2. 异常类型统一 — 分层异常体系，从 ThreeLError 继承，支持 log_when=True 自动记录
3. API 错误响应统一 — 全局拦截 + 统一 {success, error, error_type} 结构

2.2 方案选择

方案	优点	缺点	结论
A：分层异常 + 统一日志	错误可分类追踪，向后兼容，改造成本最低	少量 import 变更	✓
B：引入 Sentry 等外部监控	功能强大	需建账号、依赖外部服务、过度设计	✗
C：只加日志不改进异常	改动最小	不解决静默吞异常的根本问题	✗

2.3 设计原则

1. 渐进式改造 — 不一次改完 70+ 处 except，先搭好基础框架，逐步迁移
2. 向后兼容 — 旧 from backend.services.logger import get_logger 在新位置也有导出
3. 自记录异常 — 异常类自动记录错误级别的日志，不依赖调用方显式调用
4. 不改变业务逻辑 — 纯基础设施改造，不引入新功能

2.4 屏幕内外

- 做什么：
 - 创建 core/logger.py (从 services/logger.py 迁移)
 - 创建 core/exceptions.py (分层异常类)
 - 在 server.py 添加全局异常拦截
 - 统一 API 错误响应格式
 - 更新核心模块的 import 路径
- 不做什么：
 - ✗ 不替换 HTTP 框架
 - ✗ 不改动业务逻辑 (买点判断、信号检测等)
 - ✗ 不引入外部监控系统
 - ✗ 不一次改完 70+ 处 except (分阶段)

3. 数据模型

3.1 异常层次

```
class ThreeLError(Exception):
    """基础异常，所有自定义异常的基类"""
    def __init__(self, message, *, log_when=True, exc_info=True):
        super().__init__(message)
        if log_when:
            get_logger(self.__class__.__module__).error(
                '%s: %s', self.__class__.__name__, message,
                exc_info=exc_info or None
            )

class DataError(ThreeLError):
    """数据层异常 — 数据缺失/格式错误/过期"""
    pass

class APIError(ThreeLError):
    """API 层异常 — 参数校验/资源未找到"""
    pass

class ConfigError(ThreeLError):
    """配置异常 — 配置缺失/格式错误"""
    pass

class DataSourceError(DataError):
```

```
"""数据源异常 - 第三方 API 调用失败"""
pass
```

3.2 API 错误响应结构

```
{
    "success": false,                // boolean, 固定
    "error": "股票代码格式错误: 0000XX", // string, 人类可读错误描述
    "error_type": "APIError",        // string, 异常类型 (可选, 用于前端分类)
    "request_id": "svc-20260609-a1b2c3" // string, 请求标识 (可选, 用于日志关联)
}
```

3.3 日志文件结构

```
{LOG_DIR}/
├── 3l-server.log          # 主日志 (当前所有级别)
├── 3l-server.error.log    # 错误日志 (单独输出 ERROR+ 级别, 方便快速排查)
```

4. 系统设计

4.1 架构总览

```
server.py/wsgi.py
├── setup_logging() ← 启动时一次配置
│   ├── console (stdout) - INFO+
│   ├── file (轮转)      - INFO+
│   └── error_file       - ERROR+
```

各模块:

```
core/logger.py ← get_logger(name) 唯一入口
core/exceptions.py ← 分层异常类
```

```
server.py do_GET()/do_POST():
├── try-except 包裹整个请求处理
│   ├── APIError → 400 级响应
│   ├── DataError → 503 级响应
│   └── Exception → 500 级响应 + 日志记录
```

4.2 核心实现

core/logger.py (从 services/logger.py 迁移)

```
"""统一日志配置 - 从 services/logger.py 迁移至此"""
import logging, os, sys
from logging.handlers import RotatingFileHandler

_initialized = False

def setup_logging():
    """全局初始化日志配置 (幂等)"""
    global _initialized
    if _initialized:
        return
    from backend.config import LOG_LEVEL, LOG_DIR
    level = getattr(logging, LOG_LEVEL.upper(), logging.INFO)
    root = logging.getLogger()
    root.setLevel(level)
    # 控制台 handler
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(level)
    console.setFormatter(logging.Formatter(
```

```

        '%(asctime)s [% (levelname)s] %(name)s: %(message)s',
        '%Y-%m-%d %H:%M:%S'
    ))
    root.addHandler(console)
# 文件 handler (轮转)
try:
    os.makedirs(LOG_DIR, exist_ok=True)
    file_handler = RotatingFileHandler(
        os.path.join(LOG_DIR, '3l-server.log'),
        maxBytes=10 * 1024 * 1024, backupCount=5, encoding='utf-8'
    )
    file_handler.setLevel(level)
    file_handler.setFormatter(logging.Formatter(
        '%(asctime)s [% (levelname)s] %(name)s: %(lineno)d: %(message)s',
        '%Y-%m-%d %H:%M:%S'
    ))
    root.addHandler(file_handler)
# 独立 error 日志
err_handler = RotatingFileHandler(
    os.path.join(LOG_DIR, '3l-server.error.log'),
    maxBytes=10 * 1024 * 1024, backupCount=5, encoding='utf-8'
)
err_handler.setLevel(logging.ERROR)
err_handler.setFormatter(logging.Formatter(
    '%(asctime)s [% (levelname)s] %(name)s: %(lineno)d: %(message)s',
    '%Y-%m-%d %H:%M:%S'
))
root.addHandler(err_handler)
except (OSError, PermissionError) as e:
    root.warning('无法创建日志目录 %s: %s', LOG_DIR, e)
_initialized = True
root.info('日志初始化完成 (level=%s, dir=%s)', LOG_LEVEL, LOG_DIR)

def get_logger(name):
    if not _initialized:
        setup_logging()
    return logging.getLogger(name)

```

core/exceptions.py

```

"""统一异常层次"""
from backend.core.logger import get_logger

class ThreeLError(Exception):
    def __init__(self, message, *, log_when=True, exc_info=True):
        super().__init__(message)
        if log_when:
            get_logger(self.__class__.__module__).error(
                '%s: %s', self.__class__.__name__, message,
                exc_info=exc_info or None
            )

class DataError(ThreeLError): pass
class APIError(ThreeLError): pass
class ConfigError(ThreeLError): pass
class DataSourceError(DataError): pass

```

server.py 全局异常拦截

```
def do_GET(self):
    try:
        # ... 现有全部逻辑 ...
    except APIError as e:
        log.error('API错误: %s', e)
        self.send_json({'success': False, 'error': str(e), 'error_type': 'APIError'}, 400)
    except DataError as e:
        log.error('数据错误: %s', e)
        self.send_json({'success': False, 'error': str(e), 'error_type': 'DataError'}, 503)
    except Exception as e:
        log.exception('未捕获异常')
        self.send_json({'success': False, 'error': '服务器内部错误'}, 500)
```

4.3 文件清单

新增：

```
server/backend/core/logger.py      ← 从 services/logger.py 迁移
server/backend/core/exceptions.py  ← 新建
```

修改：

```
server/server.py                  ← import 路径变更 + 全局异常拦截
server/wsgi.py                    ← import 路径变更
server/backend/core/data_layer.py ← import 路径变更
server/backend/api/market.py      ← import 路径变更
server/backend/services/monitor_service.py ← import 路径变更
server/backend/services/watchlist_service.py ← import 路径变更
```

删除：

```
server/backend/services/logger.py ← 迁移完成后删除
```

测试新增：

```
server/backend/tests/test_core_exceptions.py ← 分层异常测试
server/backend/tests/test_core_logger.py     ← 日志功能测试
```

4.4 迁移策略（分三阶段）

阶段一（基础框架）：创建 core/logger.py + core/exceptions.py，核心模块切换 import 路径。services/logger.py 保留为兼容代理（from backend.services.logger import get_logger 继续可用）。

阶段二（全局拦截）：在 server.py 添加 try-except 包裹 + 统一错误响应。核心 API handler 改为显式 raise 分层异常。

阶段三（逐步清理）：逐步替换 70+ 处 except Exception: pass 为：

```
except DataError as e:
    log.warning('数据处理跳过: %s', e)
    # 或
except Exception as e:
    log.error('未知错误: %s', e, exc_info=True)
    raise
```

5. 执行计划

详见：[统一异常处理与日志模块 — 执行计划](#)

6. 附录

6.1 替代方案

方案	被否原因
引入 loguru 库	额外依赖，改造成本高，当前 logging 足够
引入 Sentry	外部服务 + 配置复杂度，当前阶段过度设计

6.2 开放问题

- ☐ 阶段三的 70+ 处 except 是否需要一次性改造？或分批随功能迭代逐步清理？
- ☐ request_id 是否需要在当前阶段实现？或等后续需要时再加？

6.3 影响分析摘要

文件	影响类型	变更内容
server.py	import + 逻辑	services.logger → core.logger，新增全局 try-except
wsgi.py	import	services.logger → core.logger
data_layer.py	import	延迟导入路径变更
market.py	import	services.logger → core.logger
monitor_service.py	import	services.logger → core.logger
watchlist_service.py	import	services.logger → core.logger
services/logger.py	删除	迁移完成后删除
9 个文件	usage	从 import logging 逐步迁移到 get_logger
70+ 处	except	逐步替换为分层异常处理

6.4 变更日志

版本	日期	变更内容
v1	2026-06-09	初稿